



UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master Degree in Computer Science
Curriculum Artificial Intelligence

Hebbian Co-Design of Morphing Winged Drones

Supervisors:

Prof. Andrea Cossu

Andrea Vicari, PhD Cand.

Dr. Fuda Van Diggelen

Prof. Dario Floreano

Candidate:

Salvatore Mastrangelo

ACADEMIC YEAR 2025/2026

Dedica

Glossary

- **PCA:** Principal Component Analysis
- **BPTT:** Backpropagation Through Time
- **LSTM:** Long Short-Term Memory Networks
- **MDP:** Markov Decision Process
- **POMDP:** Partially Observable Markov Decision Process
- **RL:** Reinforcement Learning
- **PPO:** Proximal Policy Optimization
- **AC-RL:** Actor-Critic Reinforcement Learning
- **EA:** Evolutionary Algorithms
- **ES:** Evolution Strategy
- **CMA-ES:** Covariance Matrix Adaptation Evolution Strategy
- **NSGA-II:** Non-dominated Sorting Genetic Algorithm II
- **EPANN:** Evolved Plastic Artificial Neural Network

Contents

1	Introduction	5
2	Technical Background & Related Work	6
2.1	Deep Reinforcement Learning	6
2.1.1	Markov Decision Processes	6
2.1.2	Actor-Critic Reinforcement Learning	8
2.1.3	Proximal Policy Optimization	10
2.2	Long Short-Term Memory Networks	11
2.2.1	Recurrent Neural Networks	11
2.2.2	Vanishing Gradient Problem	11
2.2.3	Long Short-Term Memory	11
2.3	Evolutionary Algorithms	13
2.3.1	CMA-ES	14
2.3.2	NSGA-II	16
2.4	Hebbian Learning	18
2.4.1	Hebb's Postulate	18
2.4.2	The Generalized Hebbian Rule	19
2.4.3	Plasticity Rules as the Object of Optimization	20
2.5	Related Work	21
2.5.1	Co-Design of Morphology and Control	21
2.5.2	Evolved and Learned Plasticity	23
2.5.3	Positioning of This Thesis	25
3	Problems	26
3.1	The Sequential Design Problem	26
3.2	The Morphology Optimization Problem	26
3.3	The Computational Feasibility Problem	26
4	Simulation Environment	27
4.1	Genesis: The Simulator	27
4.1.1	Advantages and Disadvantages	27
4.1.2	Physics Simulation	27

4.1.3	Aerodynamics Simulation	27
4.2	Flight Environment	27
4.2.1	Uniform Forests	27
4.2.2	Growing Forests	27
4.2.3	Latin Hypercube Forests	27
4.3	Drone Model	27
4.3.1	Drone Shape and Actuators	27
4.3.2	Drone Genome	27
4.3.3	The Standard Drone	27
5	Methodology	28
5.1	Base Controllers	28
5.1.1	Specialist LSTM Controller	28
5.1.2	Generalist LSTM Controller	28
5.1.3	Training the Generalist LSTM Controller	28
5.2	The Hebbian Controller	28
5.2.1	Hebbian Rules on the Output Layer	28
5.2.2	Weight Decay and Reset Semantics	28
5.3	Inner Loop: Evolving the Hebbian Rules	28
5.4	Outer Loop: Evolving the Drone Morphology	28
5.5	The Full Co-Design Pipeline	28
6	Results and Experiments	29
6.1	Inner Loop Experiments	29
6.1.1	Recovering Performance on Generalist Controllers	29
6.1.2	Improving Performance on Specialist Controllers	29
6.2	Full Co-Design Experiments	29
6.2.1	Co-Design vs. Morphology-Only Evolution	29
6.2.2	Swapping the Pareto Fronts	29
6.2.3	PCA Analysis of the Evolved Morphologies	29
6.2.4	Morphology Choices of The Evolution	29
6.2.5	Generalization Capabilities of Evolved Controllers	29
6.2.6	Convergence or Plasticity?	29
7	Conclusions & Future Work	30
7.1	Conclusions	30
7.2	Future Work	30
7.2.1	Meta-Learning Techniques	30
7.2.2	Testing on Real Drones	30
7.2.3	Testing on Different Systems	30

Introduction

Technical Background & Related Work

This chapter introduces the main techniques used in this work, and a series of related works that have influenced the development of the approach proposed in the following chapters. Each of the methods presented here, in addition to some novelties presented in the following chapters, is a building block of the final system.

2.1 Deep Reinforcement Learning

A common approach to train a deep neural network controller is by using Reinforcement Learning (RL): an agent interacts with an environment through a sequence of actions, and the only training signal it receives is a scalar *reward* that grades how well it is performing. RL rests on the *reward hypothesis*: that any goal can be expressed as the maximization of expected cumulative reward [37].

2.1.1 Markov Decision Processes

At each discrete time step t , the agent observes the state $s_t \in \mathcal{S}$ of the environment and selects an action $a_t \in \mathcal{A}$; the environment responds with a new state s_{t+1} and a scalar reward r_{t+1} , computed by the environment's own, generally unknown, transition and reward functions.

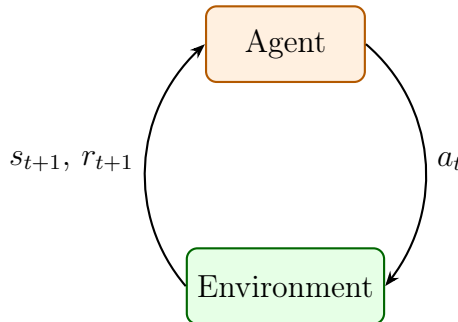


Figure 2.1: The agent-environment interaction loop. The reward r_{t+1} is a scalar with no fixed sign convention beyond larger being better: it is the only feedback the agent receives beyond a partial, possibly noisy observation of the environment.

This loop is assumed to be *Markovian*: the next state and reward depend only on the current state and action, not on the full history that produced them,

$$P(s_{t+1}, r_{t+1} \mid s_t, a_t) = P(s_{t+1}, r_{t+1} \mid s_0, a_0, r_1, \dots, s_{t-1}, a_{t-1}, r_t, s_t, a_t). \quad (2.1)$$

The Markov property is what lets the current state summarize everything relevant about the past, and is what makes the following formalism, a Markov Decision Process (MDP), tractable.

Definition 2.1.1 (Markov Decision Process). *A Markov Decision Process is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where:*

- $\mathcal{S}$ is the set of states,
- $\mathcal{A}$ is the set of actions,
- $P(s' | s, a) = P(s_{t+1}=s' | s_t=s, a_t=a)$ is the state-transition distribution,
- $R(s, a) = \mathbb{E}[r_{t+1} | s_t=s, a_t=a]$ is the expected immediate reward,
- $\gamma \in [0, 1]$ is a discount factor.

The agent's behaviour is described by a policy π , a map from states to actions: a *deterministic* policy is a function $a = \pi(s)$, while a *stochastic* policy $\pi(a | s) = P(a_t=a | s_t=s)$ is a conditional distribution over actions. Stochasticity is the source of the exploration needed to discover good actions in the first place, since a purely deterministic agent can never try anything it does not already favour. The RL objective is to find a policy that maximizes the *return*, the total discounted future reward from a given time step,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (2.2)$$

The discount factor trades off immediate against delayed reward: $\gamma \approx 0$ makes the agent myopic, $\gamma \approx 1$ far-sighted; besides this qualitative role, $\gamma < 1$ is also what keeps G_t finite for a task with no fixed episode length and bounded per-step reward.

Given a policy π , the *state-value function* $v_\pi(s)$ and *action-value function* $q_\pi(s, a)$ are the expected return obtained from a state, or from a state-action pair, when following π thereafter:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | s_t=s], \quad q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t=s, a_t=a]. \quad (2.3)$$

Because the return decomposes into an immediate reward and a discounted continuation, $G_t = r_{t+1} + \gamma G_{t+1}$, the value functions satisfy a recursive consistency condition, the *Bellman expectation equation*:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) v_\pi(s') \right) = \sum_{a \in \mathcal{A}} \pi(a | s) q_\pi(s, a). \quad (2.4)$$

An *optimal* policy π_* is one whose action-value function $q_*(s, a) = \max_\pi q_\pi(s, a)$ dominates that of every other policy in every state; since some deterministic policy always achieves q_* , namely $\pi_*(s) = \arg \max_a q_*(s, a)$, finding q_* is by itself sufficient to solve the MDP.

When $\mathcal{S}$ and $\mathcal{A}$ are small and finite and (P, R) are fully known, v_π and q_* can be computed exactly, by dynamic-programming methods such as policy iteration or value iteration, which apply the Bellman equations above as update rules until convergence (the *model-based* setting). In the *model-free* setting considered for the rest of this section, which includes every task in this work, P and R are unknown, and value functions must instead be estimated from sampled transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ obtained by actually interacting with the environment. Whenever $\mathcal{S}$, or $\mathcal{A}$, is continuous or otherwise too large to enumerate (as is the case for the sensor observations

and continuous control commands used throughout this work), a lookup table, as in tabular Q-Learning [39], is no longer an option, and v_π , q_π , or π itself must instead be represented by parametric function approximators, typically deep neural networks; this is what is meant by *deep* reinforcement learning. The remainder of this section develops the *policy-based* branch of deep RL, which parametrizes the policy directly rather than deriving it from a value function, and which is the branch this work builds on.

2.1.2 Actor-Critic Reinforcement Learning

Policy-based methods parametrize the policy itself, $\pi_\theta(a | s)$, a deep neural network with parameters θ , and optimize such parameters directly by gradient ascent on the expected return

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]. \quad (2.5)$$

Compared to deriving a policy from a value function, this tends to have better convergence properties and remains effective in high-dimensional or continuous action spaces. Its cost is that $J(\theta)$, and its gradient, can only be evaluated through samples, which tends to be high-variance and can converge to a local rather than a global optimum [37].

The *policy gradient theorem* [38] shows that, despite $J(\theta)$ depending on the environment’s unknown transition dynamics through the states π_θ itself visits, its gradient can be written without any explicit knowledge of those dynamics:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a | s) q_{\pi_\theta}(s, a)]. \quad (2.6)$$

Intuitively, $\nabla_\theta \log \pi_\theta(a | s)$ is the direction in parameter space that most increases the probability of action a in state s ; weighting it by $q_{\pi_\theta}(s, a)$ pushes θ to raise the probability of actions that lead to high return and lower that of actions that lead to low return.

Actor-critic methods reduce this variance by learning a second, separate approximation of the value function (the *critic*, e.g. $V_w(s) \approx v_{\pi_\theta}(s)$) alongside the policy, or *actor*, π_θ [20]. The critic is trained as in any value-based method, by a temporal difference (TD) update towards $r_{t+1} + \gamma V_w(s_{t+1})$, while the actor is updated by policy gradient ascent, using the critic’s own estimate of value in place of the true, unknown q_{π_θ} .

Figure 2.2 depicts both networks reading the same state s_t for simplicity, but nothing in the algorithm requires this symmetry: since only the actor π_θ is kept for deployment while the critic V_w is discarded once training ends, the critic is free to observe a wider, *privileged* state that the actor never sees: quantities that are cheap to obtain in simulation but unavailable, or unreliable, on the deployed system. Because such an asymmetric critic only ever shapes the actor’s parameter updates and never appears in its computation graph at test time, this cannot leak information the actor could not otherwise access; it simply gives the critic a less noisy value estimate to train against, which in turn lowers the variance of the advantage used to update the actor.

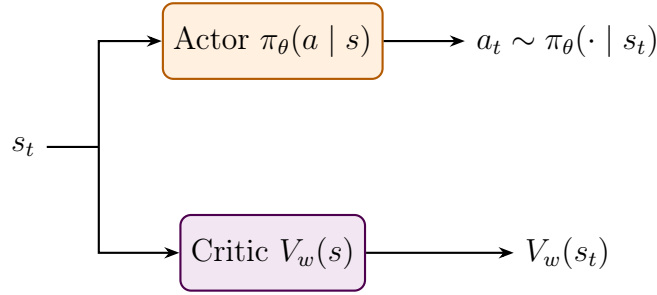


Figure 2.2: The actor-critic architecture: two networks read the same state s_t , the actor to sample an action, the critic to estimate its value $V_w(s_t)$. The TD error between $V_w(s_t)$ and $r_{t+1} + \gamma V_w(s_{t+1})$, observed once the environment responds, is what trains the critic by regression and reweights the actor’s policy-gradient update.

A further variance reduction method consists in subtracting a state-dependent baseline from the critic’s estimate without introducing any bias: since, for any fixed s , $\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a | s)] = 0$, replacing $q_{\pi_\theta}(s, a)$ in the policy gradient with the *advantage function*

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s) \quad (2.7)$$

leaves $\nabla_\theta J(\theta)$ unchanged in expectation while typically reducing its variance substantially, since $A_\pi(s, a)$ measures how much better action a is than the policy’s own average behaviour in s , rather than the absolute, and generally much larger, magnitude of the return itself. Computing $A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$ exactly would still require two separate value functions, but the single TD error $\delta_t = r_{t+1} + \gamma V_w(s_{t+1}) - V_w(s_t)$ is already, on its own, a one-sample estimate of the advantage: it substitutes the critic’s own V_w for $v_\pi(s_t)$, and the sampled $r_{t+1} + \gamma V_w(s_{t+1})$ for $q_\pi(s_t, a_t)$.

Relying on a single δ_t , however, looks only one step ahead, so the estimate inherits whatever bias V_w currently has; summing several consecutive TD errors looks further into the future and averages more of that bias away, at the cost of compounding sampling variance the further the sum reaches. Generalized Advantage Estimation (GAE) [32] controls this trade-off directly, combining successive TD errors into an exponentially-weighted sum governed by a decay parameter $\lambda \in [0, 1]$:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}. \quad (2.8)$$

GAE interpolates between the high-bias, low-variance one-step estimate $\hat{A}_t = \delta_t$ ($\lambda=0$) and the low-bias, high-variance Monte Carlo estimate obtained by summing every future TD error ($\lambda=1$). This is the same bias-variance trade-off, controlled by the same kind of exponentially-decaying trace, that $\text{TD}(\lambda)$ eligibility traces implement for the value function itself [37]. The actor and critic are typically two networks, or two heads of a partly shared network, trained jointly end to end: the critic by regression towards its TD target, the actor by advantage-weighted policy gradient ascent.

2.1.3 Proximal Policy Optimization

The actor-critic parameter update above does not, by itself, say how large a step to take. Each batch of collected experience is only informative about the policy $\pi_{\theta_{\text{old}}}$ that generated it, yet the gradient estimate computed from it is noisy; an update that moves θ too far from θ_{old} because of the strength of that noisy estimate can push π_{θ} into a region of the parameter space the batch says nothing reliable about, collapsing performance in a way that plain gradient ascent has no mechanism to prevent. Trust Region Policy Optimization (TRPO) [31] addresses this directly, by constraining each update to a region, measured by KL divergence between π_{θ} and $\pi_{\theta_{\text{old}}}$, within which the current batch is trusted to be informative, at the cost of a second-order constrained-optimization step that is expensive computationally.

Proximal Policy Optimization (PPO) [33] achieves a similar effect with an unconstrained, first-order objective, optimizable directly by ordinary stochastic gradient ascent, which is what has made it, in practice, a default choice for on-policy deep RL. Let

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (2.9)$$

be the probability ratio between the policy being optimized and the fixed, older policy that generated the batch (so that $r_t(\theta_{\text{old}}) = 1$), and let $\hat{A}_t$ be an advantage estimate, e.g. GAE. The plain policy-gradient surrogate $\mathbb{E}_t[r_t(\theta)\hat{A}_t]$ has the correct gradient at $\theta = \theta_{\text{old}}$, but nothing stops $r_t(\theta)$ from drifting arbitrarily far from 1 once optimized repeatedly on the same batch. PPO instead maximizes the *clipped surrogate objective*

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.10)$$

where ϵ , typically between 0.1 and 0.2, sets the width of the trust region. When $\hat{A}_t > 0$, the clip caps the objective's growth once $r_t(\theta)$ exceeds $1 + \epsilon$, removing any incentive to keep increasing $\pi_{\theta}(a_t | s_t)$ further on this particular batch; symmetrically, when $\hat{A}_t < 0$, it caps the objective once $r_t(\theta)$ falls below $1 - \epsilon$. The outer minimum makes L^{CLIP} a pessimistic (lower) bound on the unclipped surrogate, so clipping only ever activates in the direction that would otherwise make the update too aggressive, never in the direction that would make it too conservative. In practice, this first-order clipping behaves much like TRPO's explicit trust region, at a fraction of its implementation and computational cost.

The actor and critic are trained jointly, by combining L^{CLIP} with a value-function regression loss for the critic, $L^{\text{VF}}(\theta) = (V_{\theta}(s_t) - (\hat{A}_t + V_{\theta_{\text{old}}}(s_t)))^2$, and an entropy bonus $S[\pi_{\theta}](s_t)$ that discourages the policy from collapsing prematurely onto a deterministic one, into a single objective

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 S[\pi_{\theta}](s_t), \quad (2.11)$$

maximized by a handful of epochs of minibatch stochastic gradient ascent over each freshly collected batch of trajectories [33]. Once these epochs are done the batch is discarded, $\theta_{\text{old}} \leftarrow \theta$ is refreshed, and a new batch is collected with the updated policy: an on-policy training loop that strictly alternates environment interaction under a fixed policy with optimization of that same policy on the data it just produced.

2.2 Long Short-Term Memory Networks

2.2.1 Recurrent Neural Networks

Since the state-describing vectors are often inaccessible to a controller, and the only accessible knowledge is the sequence of observations, such settings are formally *Partially Observable Markov Decision Processes* (POMDPs) [19]. A recurrent neural network (RNN) bridges this gap by maintaining an internal state that captures information about the past. It processes a sequence $x_1, \dots, x_T$, that might represent a time series or a sequence of observations itself, and maintains a state h_t that is updated at every step from the current input and the previous state [8]:

$$h_t = g(Uh_{t-1} + Wx_t + b), \quad h_t \in \mathbb{R}^{d_h}, \quad x_t \in \mathbb{R}^{d_{in}}, \quad U \in \mathbb{R}^{d_h \times d_h}, \quad W \in \mathbb{R}^{d_h \times d_{in}}, \quad (2.12)$$

with g a pointwise nonlinearity (typically $\tanh$). Unrolled over time, this is a feed-forward computation with the same weights U, W shared across every step, trained end-to-end by backpropagation through time (BPTT) [40]: a forward pass computes every h_t in temporal order, and a backward pass propagates gradients from the last step back to the first, through the same recurrent weight matrix U at every step.

2.2.2 Vanishing Gradient Problem

This repeated multiplication by U is also the vanilla RNN's main weakness. Informally, the gradient of the loss at step T with respect to an early state h_t scales, over the $T - t$ intervening steps, with a product of $T - t$ Jacobians of the state-transition function, each dominated by U ; if the dominant eigenvalue of U is below 1 this product shrinks geometrically with $T - t$ (*vanishing gradient*), and if it is above 1 it grows geometrically (*exploding gradient*) [27]. In the vanishing case, the influence of inputs far in the past disappears from the state before training can pick up on it, which makes vanilla RNNs difficult to train on long-range dependencies. A first fix is to make the state update a convex combination, $h_t = \alpha h_{t-1} + (1 - \alpha) \tanh(Uh_{t-1} + Wx_t)$, so that a fraction α of the state is carried over unchanged instead of being fully overwritten at every step; the LSTM (below) can be read as turning this fixed, hand-set α into a learned, input-dependent, per-dimension quantity.

2.2.3 Long Short-Term Memory

Long Short-Term Memory (LSTM) networks [18] address the vanishing/exploding gradient problem by keeping two separate pieces of recurrent state instead of one, and by making the recurrence of the long-term one additive rather than a repeated matrix multiplication:

- the *hidden state* $h_t \in \mathbb{R}^{d_h}$ plays the same role as the state of a vanilla RNN: it is what is read out for prediction or further processing at step t ;
- the *cell state* $c_t \in \mathbb{R}^{d_h}$ is an internal memory that carries information across time-steps, and is only ever combined with the rest of the network through

an elementwise multiplication and an elementwise addition, never a matrix product. This is what keeps the gradient with respect to c_t from vanishing or exploding as it is backpropagated through time.

What is written into c_t , what is dropped from it, and what part of it is exposed through h_t is decided at every step by three *gates*.

Definition 2.2.1 (Gate). *A gate is a standard affine layer of h_{t-1} and x_t followed by a sigmoid activation, used to modulate a signal by elementwise multiplication:*

$$\gamma_t = \sigma(U_\gamma h_{t-1} + W_\gamma x_t + b_\gamma) \in (0, 1)^{d_h}, \quad (2.13)$$

so that, for any signal $v \in \mathbb{R}^{d_h}$, $\gamma_t \odot v$ lets a component of v through unchanged where $\gamma_t \approx 1$ and suppresses it where $\gamma_t \approx 0$.

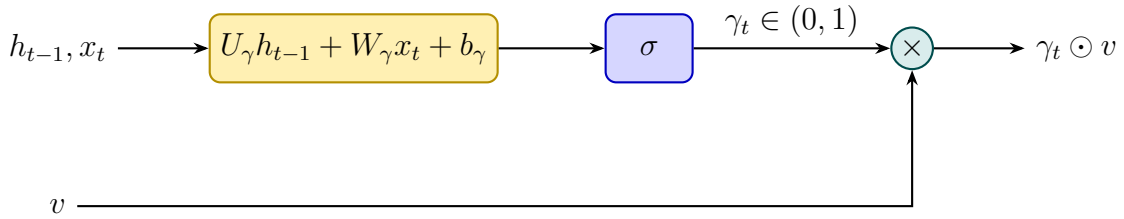


Figure 2.3: A gate: an affine map of the previous hidden state and the current input, squashed to $(0, 1)$ by a sigmoid, used to modulate a signal v by elementwise multiplication.

An LSTM cell uses three such gates, each with its own parameters:

- the **forget gate** f_t decides which part of the old cell state c_{t-1} is kept;
- the **input gate** i_t decides which part of a candidate update g_t (computed exactly as a vanilla RNN state would be) is written into the cell state;
- the **output gate** o_t decides which part of the cell state is exposed as the new hidden state h_t .

$$f_t = \sigma(U_f h_{t-1} + W_f x_t + b_f) \quad \text{forget gate} \quad (2.14)$$

$$i_t = \sigma(U_i h_{t-1} + W_i x_t + b_i) \quad \text{input gate} \quad (2.15)$$

$$g_t = \tanh(U_g h_{t-1} + W_g x_t + b_g) \quad \text{candidate cell content} \quad (2.16)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad \text{cell state update} \quad (2.17)$$

$$o_t = \sigma(U_o h_{t-1} + W_o x_t + b_o) \quad \text{output gate} \quad (2.18)$$

$$h_t = o_t \odot \tanh(c_t) \quad \text{hidden state update} \quad (2.19)$$

where $\odot$ denotes the elementwise product. Each gate, and the candidate g_t , has its own weight matrices $U_\square$, $W_\square$ and bias $b_\square$, learned jointly with the rest of the network: a single LSTM layer of hidden size d_h therefore holds four sets of parameters shaped like a single vanilla-RNN layer's, one each for f_t , i_t , g_t , o_t .

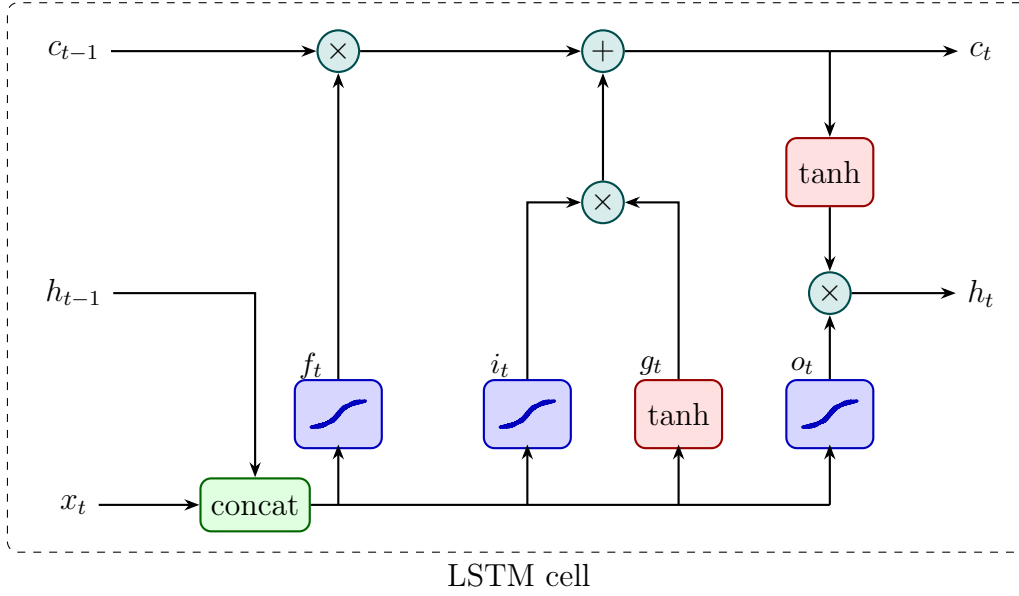


Figure 2.4: Structure of an LSTM cell. The top rail carries the cell state c_t across time-steps through only an elementwise multiplication (forget gate f_t) and an elementwise addition (gated candidate $i_t \odot g_t$), with no matrix product in between: this is the property that keeps gradients from vanishing or exploding along this path. The hidden state h_t is a gated, squashed read-out of c_t , and h_{t-1}, x_t jointly drive all four gates/candidate at the bottom.

Figure 2.4 makes explicit why this design addresses the problem laid out in the previous section: the path carrying c_t across time-steps involves only elementwise operations gated by f_t and $i_t \odot g_t$, with no repeated multiplication by a shared weight matrix. Backpropagating through this path therefore does not force the gradient to contract or expand geometrically with the number of time-steps, which is what allows an LSTM to learn dependencies over much longer sequences than a vanilla RNN.

2.3 Evolutionary Algorithms

Evolutionary Algorithms (EAs) are a family of population-based, gradient-free optimization methods loosely inspired by natural selection [7]. Where a gradient method follows the slope of the objective from a single point, an EA maintains a whole *population* of candidate solutions, or *individuals*, each encoded by a vector of parameters called its *genome*, and repeats a simple loop over successive *generations*:

1. *Evaluation*: every individual is assigned a *fitness*, a scalar score obtained by actually running it on the task;
2. *Selection*: the fitter individuals are chosen as *parents*, so that good genomes get to reproduce more than bad ones;
3. *Variation*: new individuals, the *offspring*, are produced from the parents by

mutation (a random perturbation of a single genome) and *recombination*, or *crossover*, which mixes the genomes of two parents;

4. *Replacement*: the offspring form the next population, possibly alongside the best individuals of the current one (*elitism*), so that a good solution, once found, is never lost.

Over many generations, selection pressure concentrates the population in high-fitness regions of the search space, while variation keeps exploring around it.

The appeal of this recipe is that it treats the objective as a black box: nothing is assumed about it beyond the ability to evaluate it. It needs no gradient, no differentiability, and no smoothness, and it is indifferent to whether the genome is continuous, discrete, or mixed. This makes EAs the natural tool whenever the quantity being optimized is only defined through a complete simulation and, more generally, whenever gradient-based training, such as the RL methods of the previous section, cannot reach the parameters of interest. The price is sample efficiency: every generation costs as many full evaluations as there are individuals, which is affordable mainly when evaluations can be run in parallel, as is the case for GPU-batched simulation. Two EAs are used in this work, one for each of the two settings they were designed for: CMA-ES for single-objective optimization over continuous parameters, and NSGA-II for problems with several competing objectives.

2.3.1 CMA-ES

Evolution Strategies (ES) are the branch of EAs specialized to continuous search spaces, $x \in \mathbb{R}^n$. In an ES, variation is not a discrete crossover but a Gaussian perturbation: at every generation, λ offspring are sampled around a single mean vector m ,

$$x_k = m + \sigma z_k, \quad z_k \sim \mathcal{N}(0, C), \quad k = 1, \dots, \lambda, \quad (2.20)$$

where $\sigma > 0$ is a global *step size* and $C \in \mathbb{R}^{n \times n}$ is a covariance matrix that shapes the cloud of samples. The offspring are evaluated and sorted by fitness, and the mean is moved to a weighted average of the best $\mu < \lambda$ of them,

$$m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda}, \quad w_1 \geq w_2 \geq \dots \geq w_{\mu} > 0, \quad \sum_{i=1}^{\mu} w_i = 1, \quad (2.21)$$

where $x_{i:\lambda}$ denotes the i -th best of the λ samples. Note that only the *ranking* of the fitness values enters the update, never their magnitude: the algorithm is therefore invariant to any monotone rescaling of the objective, and requires no normalization of the reward. The flip side is that, when the fitness is a noisy estimate (as the outcome of a stochastic simulation is), what degrades the update is precisely noise large enough to reorder the candidates, and the standard remedies are to enlarge λ or to average each individual's fitness over repeated evaluations.

With σ and C fixed, this is a very simple algorithm, whose performance depends entirely on whether the cloud of samples happens to match the local shape of the fitness landscape: a narrow, tilted valley, for instance, forces an isotropic Gaussian to take tiny steps, since most large steps fall off the valley walls. The Covariance

Matrix Adaptation Evolution Strategy (CMA-ES) [16, 15] removes this dependence by adapting both σ and C online, from the history of the steps that turned out to be successful, so that the sampling distribution itself learns the shape of the landscape; Figure 2.5 illustrates the effect.

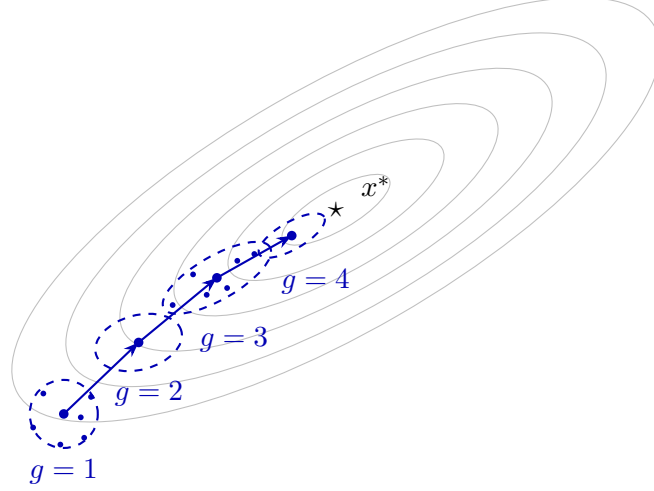


Figure 2.5: Covariance adaptation in CMA-ES on a fitness landscape shaped like a narrow, tilted valley (grey contours; the optimum is x^*). Each dashed ellipse is the one-standard-deviation contour of the sampling distribution $\mathcal{N}(m, \sigma^2 C)$ at generation g , and the dots are its samples. The search starts isotropic ($C = I$), but the successful samples of each generation lie preferentially along the valley, and the covariance update progressively stretches the distribution in that direction: by $g = 3$ the algorithm is taking long steps along the valley and short ones across it, which an isotropic search could never do.

The covariance is updated, after every generation, towards the directions in which the selected offspring were found. Writing $y_{i:\lambda} = (x_{i:\lambda} - m)/\sigma$ for the normalized step that produced the i -th best sample, the update reads

$$C \leftarrow (1 - c_1 - c_\mu) C + c_1 p_c p_c^\top + c_\mu \sum_{i=1}^{\mu} w_i y_{i:\lambda} y_{i:\lambda}^\top, \quad (2.22)$$

with small learning rates $c_1, c_\mu \in (0, 1)$. The last term is the weighted empirical covariance of this generation's successful steps: it stretches the distribution along the directions the good samples came from, and shrinks it along the others. The middle term does the same across generations, through the *evolution path* p_c , an exponentially-decaying running sum of the successive displacements of the mean: a direction in which m keeps moving, generation after generation, is amplified, while directions in which it merely oscillates cancel out. On a smooth objective the net effect is that C comes to approximate the inverse Hessian near the optimum, so that CMA-ES behaves much like a second-order method, without ever computing a derivative.

The step size is adapted separately, by *cumulative step-size adaptation*: a second evolution path accumulates the displacements of the mean, rescaled by $C^{-1/2}$ so that, if selection were purely random, its length would be that of a random walk. If

consecutive steps point in a consistent direction the path grows longer than a random walk would, which signals that the steps are too short and σ is increased; if they largely cancel out, the path is shorter than expected and σ is decreased. This lets the algorithm take large steps while far from the optimum and contract progressively as it closes in on it. The learning rates, the weights w_i , and the population size $\lambda = 4 + \lfloor 3 \ln n \rfloor$ all have default values that work robustly across problems, so that the number of parameters the user must provide is reduced to just the initial mean $m^{(0)}$ and step size $\sigma^{(0)}$: this near absence of tuning is a large part of why CMA-ES is the default choice for black-box optimization in moderate dimensions.

2.3.2 NSGA-II

Many design problems do not have a single objective. A plane should travel far *and* consume little energy; a structure should be light *and* stiff. Typically such objectives conflict, so that improving one worsens the other, and no single solution is best in every respect. *Multi-objective optimization* makes this explicit by asking to minimize a vector of M objectives at once, $f(x) = (f_1(x), \dots, f_M(x))$, where an objective to be maximized is handled simply by negating it. Since there is no natural total order among vectors, the notion of “better” has to be replaced by a partial one:

Definition 2.3.1 (Pareto dominance). *A solution x dominates a solution x' , written $x \prec x'$, if it is no worse in every objective and strictly better in at least one:*

$$f_i(x) \leq f_i(x') \quad \forall i \in \{1, \dots, M\} \quad \text{and} \quad f_j(x) < f_j(x') \quad \text{for some } j. \quad (2.23)$$

The representation of such dominance can be visualized in a two-dimensional space in Figure 2.6(a). A solution that is dominated by no other is *Pareto-optimal*; the set of all such solutions is the *Pareto set*, and its image in objective space is the *Pareto front*. Every point on the front is a trade-off that cannot be improved in one objective without giving something up in another, and which of them is “best” is not a question the optimizer can answer: it is left to the designer, once the front is known. The goal of a multi-objective optimizer is therefore not one point but an approximation of the whole front: a set of solutions that lies close to it (*convergence*) and is spread evenly along it (*diversity*). One could obtain this by fixing a weighted sum of the objectives and running a single-objective optimizer once per choice of weights, but that requires choosing the weights in advance, and a weighted sum cannot reach the points of a front where it bends inwards (its non-convex parts). A population-based method, in contrast, already maintains a set of solutions, and can be steered to spread that set along the front within a single run; this is why EAs are the dominant approach to multi-objective optimization.

The Non-dominated Sorting Genetic Algorithm II (NSGA-II) [4] is the most widely used such method. It is a genetic algorithm in the sense of section 2.3: for real-valued genomes, the standard variation operators are simulated binary crossover (SBX) and polynomial mutation [3], both of which produce offspring in the neighbourhood of their parents. What distinguishes it is the selection step, which is built on two ingredients, both illustrated in Figure 2.6(b).

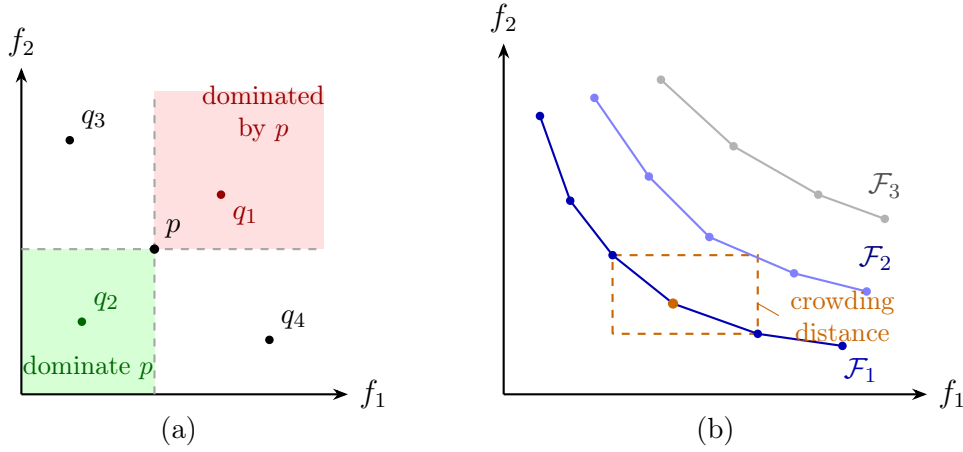


Figure 2.6: Two objectives, both to be minimized. (a) Pareto dominance: p dominates every point in the shaded red region (q_1), is dominated by every point in the green one (q_2), and is *incomparable* with points such as q_3 and q_4 , each of which is better than p in one objective and worse in the other. (b) Non-dominated sorting of a population into fronts $\mathcal{F}_1 \prec \mathcal{F}_2 \prec \mathcal{F}_3$: $\mathcal{F}_1$ contains the points dominated by no other, $\mathcal{F}_2$ those dominated only by members of $\mathcal{F}_1$, and so on. The dashed box is the crowding cuboid of the orange point of $\mathcal{F}_1$: the rectangle spanned by its nearest neighbours along each objective, whose side lengths sum to its crowding distance. A point with a large cuboid sits in a sparsely populated part of its front, and is preferred over one in a crowded region.

The first is *non-dominated sorting*, which partitions the population into a sequence of *fronts* $\mathcal{F}_1, \mathcal{F}_2, \dots$: $\mathcal{F}_1$ is the set of individuals that no other individual dominates, $\mathcal{F}_2$ the set of those that become non-dominated once $\mathcal{F}_1$ is removed, and so on until every individual has been assigned to a front. The index of an individual's front is its *rank*, and lower is better: $\mathcal{F}_1$ is the current approximation of the Pareto front, $\mathcal{F}_2$ the second-best layer behind it. The “fast” procedure of NSGA-II computes this partition in $O(MN^2)$ time for a population of N individuals, which is negligible next to the cost of evaluating them.

The second is the *crowding distance*, which breaks ties among individuals of the same front in favour of diversity. For each objective, the front is sorted along that objective and each individual is assigned the distance between its two neighbours, normalized by the objective's range; the crowding distance is the sum of these values over all objectives, and the two extreme individuals of each front, which have only one neighbour, receive an infinite value so that they are always kept. Geometrically, it measures the size of the largest box around an individual that contains no other member of its front: a large value marks an isolated individual in a sparsely covered region, which is exactly where the front's approximation most needs improving.

The two are combined into a single *crowded-comparison* rule: between two individuals, the one with the lower rank wins, and if the ranks are equal, the one with the larger crowding distance does. A generation of NSGA-II then proceeds as follows. Parents are chosen from the population P of size N by binary tournaments under this rule, and produce N offspring Q by crossover and mutation. Parents and

offspring are merged into a pool $P \cup Q$ of size $2N$, the pool is sorted into fronts, and the next population is filled with whole fronts in order, $\mathcal{F}_1$ first, until the next front no longer fits; that last front is truncated to the remaining slots by keeping its most isolated members, in descending order of crowding distance. Merging parents and offspring makes the algorithm elitist (a non-dominated individual can only be displaced by one that dominates it or by a more isolated peer), while the crowding distance keeps the population spread along the front without any additional parameter to tune, which is the combination that made NSGA-II the reference algorithm for problems with two or three objectives.

Since the output of a multi-objective run is a set rather than a number, comparing two runs, or tracking a single run across generations, requires a scalar measure of front quality. The most common is the *hypervolume* indicator [41]: the volume of objective space (an area, when $M = 2$) that is dominated by the front and bounded by a fixed *reference point*, chosen to be worse than every solution in every objective. A front that lies closer to the true Pareto front, or covers a wider portion of it, dominates a larger region, so the hypervolume rewards convergence and diversity at once; it is only meaningful, however, when the same reference point is used for every front being compared.

2.4 Hebbian Learning

Hebbian learning is a different, and much older, view of what learning in a neural network can be: a synaptic weight changes as a consequence of the activity that flows through it, according to a *local* rule that involves no error signal and needs no supervision, and keeps doing so for as long as the network is running.

2.4.1 Hebb's Postulate

The idea originates in neuropsychology. Hebb [17] proposed, as the mechanism by which associations form in the brain, that

When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased,

a statement usually compressed into the slogan “*cells that fire together wire together*”. The postulate says nothing about what the strengthened connection is *for*: the synapse strengthens simply because the two neurons it links have repeatedly been active at the same time.

For a rate-based neuron model (the one used by artificial neural networks), the simplest formalization considers a single postsynaptic unit with output y , receiving presynaptic activities $x \in \mathbb{R}^n$ through a weight vector $w \in \mathbb{R}^n$, $y = w^\top x$, and lets each weight change in proportion to the product of the activities at its two ends:

$$\Delta w_j = \eta x_j y, \quad j = 1, \dots, n, \quad (2.24)$$

with $\eta > 0$ a small *learning rate*. For a whole layer with input $x \in \mathbb{R}^n$, output $y \in \mathbb{R}^m$ and weight matrix $W \in \mathbb{R}^{m \times n}$, the same rule reads, in matrix form, as an outer product:

$$\Delta W = \eta y x^\top, \quad \Delta W_{ij} = \eta y_i x_j. \quad (2.25)$$

The dominant property of (2.24), shared by the whole family of rules that descends from it, is *locality*. The update of W_{ij} depends only on x_j and y_i , the two activities physically present at the synapse, and on nothing else: not on the activity of other neurons, not on the state of other layers, and not on any measure of how well the network is doing. By contrast, with backpropagation the gradient of a loss L with respect to the same weight, $\partial L / \partial W_{ij} = \delta_i x_j$, has the same presynaptic factor x_j but replaces the postsynaptic activity y_i with an error term δ_i that is computed at the output of the network and transported backwards through every downstream layer (Figure 2.7). A Hebbian update can therefore be applied at every forward pass, during the normal operation of the network.

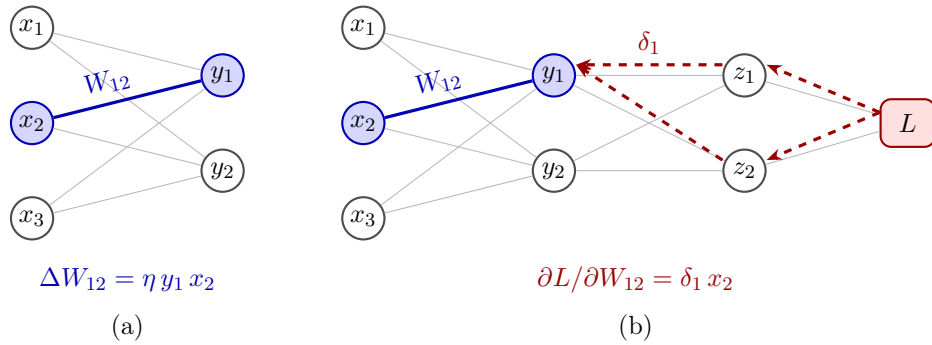


Figure 2.7: Locality of the Hebbian update. (a) Under Hebb’s rule, the change of the highlighted weight W_{12} is a function of the two activities at its ends, x_2 and y_1 , and of nothing else; it can be computed in the forward pass, at the synapse. (b) Under backpropagation, the same weight’s gradient keeps the presynaptic factor x_2 but replaces y_1 with an error δ_1 , which only exists once a loss L has been evaluated at the output and its gradient carried back (dashed) through every layer downstream of the synapse.

2.4.2 The Generalized Hebbian Rule

A local rule is any function $F(\cdot)$ of the quantities available at the synapse:

$$\Delta W_{ij} = \eta F(x_j, y_i, W_{ij})$$

Gerstner and Kistler [12] note that, for small activities, any such F can be expanded in a Taylor series around $x_j = y_i = 0$, and that truncating the expansion after the bilinear term gives the most general rule that is at most linear in each activity:

$$\Delta W_{ij} = \eta (A_{ij} x_j y_i + B_{ij} x_j + C_{ij} y_i + D_{ij}), \quad (2.26)$$

usually referred to as the *ABCD rules* [26, 36]. Each coefficient has a distinct meaning:

- A is the *correlation* term, Hebb’s rule itself. A positive A strengthens a synapse when its two ends are simultaneously active, a negative one weakens it, which decorrelates the neuron’s output from that input;
- B is the *presynaptic* term: the weight changes whenever the input is active, regardless of what the neuron does. With $B < 0$, an input that keeps firing without contributing to the output is progressively weakened, the “pre active, post silent” form of suppression;
- C is the *postsynaptic* term: the weight changes whenever the neuron fires, regardless of that particular input. With $C < 0$, every synapse of a neuron is weakened each time it fires, a normalization that redistributes strength from the inputs that stay silent to those that actually drive it;
- D is a constant drift, independent of activity: a steady creep of the weight in one direction, which can act as a bias, or, with the opposite sign to the weight, as a crude decay.

It should be noted that in case of repeated activations, the rule may lead to a runaway effect, where the weights grow without bound. This is particularly true for the A term, which can cause a positive feedback loop if the inputs are consistently active. To overcome this problem, various normalization techniques can be applied, such as weight clipping or weight decay. While weight clipping simply limits the maximum value a weight can take, weight decay introduces a small penalty to the weight values, effectively reducing them over time, in a fashion similar to regularization in supervised learning:

$$\Delta W_{ij} = \eta (A_{ij} x_j y_i + B_{ij} x_j + C_{ij} y_i + D_{ij}) - \lambda W_{ij}$$

The coefficients can be shared by an entire layer, as one rule for all the synapses, or, as written in (2.26), be a separate matrix each, of the same shape as W , so that every synapse follows its own rule [26, 25]. In the latter case a layer of $m \times n$ weights is described by $4mn$ coefficients, and the learning rate η may itself be per synapse. If η is not fixed but modulated by a dedicated signal, this is called *neuromodulation* [36]. The point of this parametrization is that the same template, with different coefficients, produces qualitatively different behaviours: a pure correlation detector, a decorrelating one, a synapse that only ever decays, one that potentiates and then forgets. Which of them is appropriate at a given synapse is not something the rule can decide, since it has no access to the task; the coefficients are the knobs through which a purpose is imposed on a rule that is otherwise blind to it.

2.4.3 Plasticity Rules as the Object of Optimization

In a *plastic* network, the weights are no longer the parameters that define the network: they are part of its state, initialized to some value at the beginning of a *lifetime*, either an episode or the deployment itself, and driven from then on by the rules, along a trajectory that depends on the inputs the network happens to receive. What stays fixed across lifetimes, and is therefore what actually defines the

network, are the rules, the rate η and the initial weights. The consequence is that the same network, placed in two different environments, ends up with two different sets of weights, each shaped by the activity that environment induced. This is the mechanism by which a plastic network can adapt, within a lifetime and without any gradient, to conditions that were not present when it was designed, and at the same time the reason a rule is not a solution: it is a procedure, which only becomes useful once its coefficients are such that the weight trajectory it produces improves the network's behaviour rather than degrading it.

Finding such coefficients is an optimization problem, with an unusual objective: the quality of a rule is the performance of the network at the end of, or throughout, a lifetime of applying it, which depends on the coefficients only through the whole sequence of weight updates and of the outputs those weights produced. This is a form of *meta-learning* in which the outer level searches over learning rules and the inner level is the lifetime of the network under one set of them. The objective can in principle be differentiated through the chain of updates [22], but when the lifetime is a closed-loop interaction with an environment, as in control, the dependence of the outcome on the coefficients is long, indirect, and only defined by running it. In such cases it is possible to treat them as a black box and search them with the evolutionary algorithms of section 2.3. Networks obtained this way, whose genome encodes rules rather than weights, are known as *evolved plastic artificial neural networks*, or EPANNs [11, 35].

2.5 Related Work

This thesis sits at the meeting point of two lines of research that have so far been pursued separately: the joint optimization of a robot's body and of its controller, and the design of neural controllers whose weights keep changing during operation under rules that are themselves the object of optimization. This section reviews the works of each line that are closest to what is done here, singling out in each case the design choice that the present work makes differently, and closes by stating where it stands with respect to both.

2.5.1 Co-Design of Morphology and Control

Designing a robot means choosing a body and a controller, and the two choices are not independent: the best controller depends on the body it drives, and the value of a body can only be judged through the best behaviour it affords. *Co-design* makes this coupling explicit by optimizing both. Writing m for a morphology in a design space $\mathcal{M}$, θ for the parameters of a controller and $F(m, \theta)$ for the performance of the closed loop they form, obtained by simulating it, the problem is a nested one:

$$m^* = \arg \max_{m \in \mathcal{M}} F(m, \theta^*(m)), \quad \theta^*(m) = \arg \max_{\theta} F(m, \theta). \quad (2.27)$$

The formulation states the problem without solving it: F is a black box, and evaluating the outer objective at a single candidate m requires solving, or approximating, the inner problem for that body. The works reviewed below differ mainly in how they pay for the inner level, and (2.27) is a convenient way to compare them.

The earliest approach collapsed the two levels into one. Sims [34] evolved virtual creatures whose genome, a directed graph, encoded at the same time the segments of the body and the neural circuits embedded in each segment, obtaining swimmers, walkers and jumpers from a single genetic algorithm; Lipson and Pollack [21] applied the same idea to machines made of bars, actuators and artificial neurons, and fabricated the evolved designs by rapid prototyping. In both cases the pair (m, θ) is a single individual and the inner problem is never solved explicitly: the controller of a body is whatever the mutations happened to pair with it.

Cheney et al. [2] identified why this becomes a problem as the design space grows. A controller is co-adapted to the body it evolved with, so a mutation of the body is almost always harmful in the short term, however promising the new body may be once a controller is newly adapted to it. Selection therefore removes morphological novelty faster than it can be exploited, and the search converges prematurely on a body found early. Their remedy, *morphological innovation protection*, temporarily lifts selection pressure from individuals whose body has just changed, giving the controller time to catch up before the body is judged. The underlying principle carries over to every method in this section: a body can only be assessed fairly once a controller has been adapted to it.

Eiben and Hart [6] turned that principle into a design rule, arguing that every newborn body should go through a learning phase before its fitness is measured. Gupta et al. [13] showed what this buys at scale. In their Deep Evolutionary Reinforcement Learning framework, bodies are kinematic trees of cylinders connected by hinge joints, a space of the order of 10^{18} morphologies; an asynchronous tournament evolution mutates limbs and joints, and the controller of every new body is trained from scratch by PPO (subsection 2.1.3) on the body's own egocentric sensors for a fixed budget of interactions, with the fitness of the body being the reward reached at the end of training. Two findings are relevant here. Bodies evolved in harder environments, uneven terrain and manipulation rather than flat ground, perform better when transferred to new tasks, which the authors read as environmental complexity fostering morphological intelligence. And although nothing selects for it explicitly, the time a controller needs to reach a given performance halves within ten generations: evolution finds bodies that are easier to control, the morphological counterpart of the Baldwin effect. What makes these results possible is also what makes the method expensive: the inner problem of (2.27) is paid in full at every evaluation, with a complete reinforcement learning run per body and no inheritance of the controller, which required more than a thousand CPUs running in parallel.

The same division of labour has recently been applied to drones. Muff et al. [24] evolve hexacopters in which each of the six arms is described by its length, two placement angles, two motor tilt angles and the spin direction of its rotor. A $(\mu + \lambda)$ evolution strategy with tournament selection acts on these morphological parameters, with repair operators that resolve collisions between arms and a static hovering test that discards unviable designs, while the controller of each morphology, a small feed-forward network, is learned from scratch by PPO on four waypoint-following tasks (circle, slalom, shuttle run and figure of eight). The evolved drones, with tilted rotors and asymmetric arms, outperform the conventional hexacopter on every task. The authors also introduce metrics for the learning process itself and

find that evolved bodies reach their learning breakthrough earlier and converge to higher rewards, confirming the Baldwin-like effect of Gupta et al. As in that work, evolution owns the morphology and reinforcement learning owns the controller, and the cost of the inner level, several hundred million simulation steps per body, is what limits the search to under a thousand morphologies.

Bergonti et al. [1], in the laboratory where this thesis was carried out, address a morphing winged drone of the same family as the one used in the present work, a fixed-wing glider whose wings carry actuated joints. The outer level is NSGA-II (subsection 2.3.2) over the wing geometry, the number and orientation of the morphing joints of each wing, the choice of servomotors and propulsion unit from commercial catalogues, and a scalar that weighs the two objectives, energy consumption and mission time over a suite of scenarios. The inner level is not a learned controller but a trajectory optimization: for every candidate design, a nonlinear program solved by direct multiple shooting computes the time series of joint torques and thrust that completes each scenario at minimum cost, subject to the multi-body dynamics with aerodynamic forces, the actuator limits, the obstacles and the checkpoints. The co-designed drones cut energy by more than a third and mission time by a fifth or more with respect to the commercial glider they start from. The choice of an open-loop optimal trajectory as inner solution is what makes tens of thousands of inner evaluations affordable on CPUs, but it assumes a known model and no disturbances, and it leaves the closed-loop controller that would fly the real drone outside the optimization.

A different way to escape the cost of the inner level is to amortize it. Gupta et al. [14] train a single Transformer policy, MetaMorph, on a large set of bodies at once, encoding each limb as a token that combines its proprioceptive readings with a description of its geometry and joints, so that the policy reads the morphology it is controlling as one more input modality. The result generalizes zero-shot to unseen variations of kinematics and dynamics and transfers to new bodies with little additional training. In the terms of (2.27), $\theta^*(m)$ is replaced by a single θ for a policy $\pi(a \mid o, m)$ conditioned on m : the inner problem is paid once, for the whole design space, at the price of telling the controller which body it has.

2.5.2 Evolved and Learned Plasticity

The idea of letting evolution choose the learning rule rather than the weights, introduced in subsection 2.4.3, also has a history in the laboratory where this work was done. Floreano and Urzelai [11] evolved controllers for a Khepera robot in which the genome specified, for each synapse, which of four Hebbian rules it obeyed and at what rate, while the weights themselves were drawn at random at the beginning of every life and shaped only by the rule. On a task that required a sequence of behaviours, these controllers not only learned it within a life but, once evolved in simulation, transferred to the physical robot, and to a larger robot with a different body, better than controllers with evolved fixed weights. The review by Soltoggio, Stanley and Risi [35] consolidates two decades of such EPANNs and draws from them one lesson that matters for the present work: plasticity is selected only when something varies, between or within lifetimes, that a fixed genome cannot antici-

pate. On a stationary task evolution finds it cheaper to write the solution into fixed weights, and a plastic mechanism that is never needed is at best neutral and often harmful. Neuromodulation, the gating of the learning rate by a dedicated signal, is the other recurring theme of that literature [36].

Miconi, Stanley and Clune [22] took the opposite route and made the plasticity differentiable. Each connection carries a fixed weight W_{ij} and a Hebbian trace H_{ij} whose contribution is scaled by a coefficient α_{ij} :

$$y_i = \sigma\left(\sum_j (W_{ij} + \alpha_{ij} H_{ij}) x_j\right), \quad H_{ij} \leftarrow (1 - \eta) H_{ij} + \eta x_j y_i, \quad (2.28)$$

and W , α and η are all trained by gradient descent, backpropagating through the entire lifetime of the network, trace updates included. The decomposition is instructive: the weight is split into a part that is learned once and a part that is written by experience and forgotten geometrically, and α decides, synapse by synapse, how much of the second part the network is allowed. Plastic networks trained this way outperformed non-plastic recurrent networks on a pattern memorization task and competed with dedicated architectures on few-shot image classification, and a later extension, Backpropamine [23], replaced the fixed η by a neuromodulatory signal computed by the network itself. The price of differentiability is that the lifetime must be differentiable: the base weights and the rule are co-adapted by the same gradient, and the method does not apply when the lifetime is a closed-loop interaction with a physics simulator, which is why the works below, and this thesis, return to evolution.

Najarro and Risi [25] did exactly that, with the parametrization of (2.26) extended by a learning rate per synapse, so that every connection is described by five coefficients, and a natural evolution strategy [30] searching them. The weights of the network are redrawn uniformly at random at the start of every episode and organize themselves under the rules within a few dozen time steps, without any reward signal during the lifetime. On a simulated quadruped this produced the result that has since become the reference for the approach: a Hebbian network evolved on the intact body and on one damaged leg keeps walking when a leg it has never seen damaged is removed, where a network with evolved fixed weights falls. The same experiment also shows the cost: on the intact body the fixed-weight network walks farther than the plastic one. Plasticity buys robustness to a change of body at a price on the nominal body, and the random initialization at each episode is what forces the rules to do the whole job of building the controller.

Two later works ask where and how many rules are needed. Pedersen and Risi [28] evolve the rules of the same quadruped and periodically merge them by clustering, so that many synapses come to share one rule; the number of evolved coefficients drops by more than an order of magnitude and the smaller set generalizes better to unseen damage, a computational version of the genomic bottleneck. Ferigo et al. [9] evolve one rule per synapse in the controllers of voxel-based soft robots, whose body deforms as it moves, and report that the Hebbian controllers match or exceed static ones in locomotion and adapt to damage that was never seen during evolution.

Van Diggelen et al. [5] bring evolved Hebbian rules to swarm robotics. Each robot carries a small memoryless network with a rule of the form (2.26) on every synapse

and a fixed learning rate, and a single set of rule coefficients, evolved by CMA-ES (subsection 2.3.1) on the performance of the whole swarm, is shared by all its members. At the beginning of each trial every robot draws its own weights at random; since each robot then experiences a different sensory history, identical rules drive the weights of different robots to different places, and heterogeneous roles, including collective behavioural switching, emerge from a homogeneous genome. The rules outperform multi-agent reinforcement learning baselines on standard benchmarks and transfer from simulation to ten physical robots with a loss of under one percent. Two features of this design recur throughout the plasticity literature reviewed here: the controller converges from a random initialization at every trial, and the same rule set is used by every agent regardless of its individual situation.

2.5.3 Positioning of This Thesis

Read through (2.27), the co-design works above pay for the inner level in one of three ways:

1. *in full*, with a reinforcement learning run per body [13, 24];
2. *exactly solved*, but for an open-loop trajectory, which leaves the closed-loop controller out of the design [1];
3. *once*, with a policy that must be told which body it controls [14].

The plasticity works, for their part, all build the controller from random weights at every episode [11, 25, 9, 5] or co-train the base weights and the rule by the same gradient [22]; none places evolved rules on top of a policy that was pretrained by reinforcement learning and then frozen, and none uses plasticity across a population of bodies inside a co-design loop.

This thesis combines the two lines in a way that addresses both gaps. A single recurrent controller is trained once by reinforcement learning over a distribution of morphologies, without being told which body it is flying, and is then frozen. Hebbian rules of the form (2.26) act only on its output layer, so that the plastic part of the controller is, as in (2.28), an experience-driven departure from a fixed part, with two differences: the fixed part is learned by reinforcement learning and the rules by evolution, and the departure starts from the frozen weights rather than from random ones. The morphology is then evolved by NSGA-II with the inner problem reduced to running the frozen controller and its rules on the candidate body, so that evaluating a body costs rollouts rather than a training run. Two results from the literature support these choices. Raghu et al. [29] showed that in gradient-based meta-learning [10] almost all of the adaptation happens in the last layer, while the layers below are reused as fixed features; restricting plasticity to the output layer follows the same division of labour. And the lesson of Soltoggio et al. [35] indicates what the varying quantity must be for plasticity to be worth selecting: here it is the body itself, which changes from one lifetime to the next and which the controller is never told. The method is described in chapter 5.

Problems

3.1 The Sequential Design Problem

3.2 The Morphology Optimization Problem

3.3 The Computational Feasibility Problem

Simulation Environment

4.1 Genesis: The Simulator

4.1.1 Advantages and Disadvantages

4.1.2 Physics Simulation

4.1.3 Aerodynamics Simulation

4.2 Flight Environment

4.2.1 Uniform Forests

4.2.2 Growing Forests

4.2.3 Latin Hypercube Forests

4.3 Drone Model

4.3.1 Drone Shape and Actuators

4.3.2 Drone Genome

4.3.3 The Standard Drone

Methodology

5.1 Base Controllers

5.1.1 Specialist LSTM Controller

5.1.2 Generalist LSTM Controller

5.1.3 Training the Generalist LSTM Controller

5.2 The Hebbian Controller

5.2.1 Hebbian Rules on the Output Layer

5.2.2 Weight Decay and Reset Semantics

5.3 Inner Loop: Evolving the Hebbian Rules

5.4 Outer Loop: Evolving the Drone Morphology

5.5 The Full Co-Design Pipeline

Results and Experiments

6.1 Inner Loop Experiments

6.1.1 Recovering Performance on Generalist Controllers

6.1.2 Improving Performance on Specialist Controllers

6.2 Full Co-Design Experiments

6.2.1 Co-Design vs. Morphology-Only Evolution

6.2.2 Swapping the Pareto Fronts

6.2.3 PCA Analysis of the Evolved Morphologies

6.2.4 Morphology Choices of The Evolution

6.2.5 Generalization Capabilities of Evolved Controllers

6.2.6 Convergence or Plasticity?

Conclusions & Future Work

7.1 Conclusions

7.2 Future Work

7.2.1 Meta-Learning Techniques

7.2.2 Testing on Real Drones

7.2.3 Testing on Different Systems

List of Figures

2.1	The agent-environment interaction loop	6
2.2	Actor and critic networks	9
2.3	A LSTM gate	12
2.4	Structure of an LSTM cell	13
2.5	Covariance adaptation in CMA-ES	15
2.6	Pareto dominance, non-dominated fronts and crowding distance . . .	17
2.7	Locality of the Hebbian update	19

Bibliography

- [1] Fabio Bergonti et al. “Co-Design Optimisation of Morphing Topology and Control of Winged Drones”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2024, pp. 8679–8685.
- [2] Nick Cheney et al. “Scalable Co-Optimization of Morphology and Control in Embodied Machines”. In: *Journal of The Royal Society Interface* 15.143 (2018), p. 20170937.
- [3] Kalyanmoy Deb and Ram Bhushan Agrawal. “Simulated Binary Crossover for Continuous Search Space”. In: *Complex Systems* 9.2 (1995), pp. 115–148.
- [4] Kalyanmoy Deb et al. “A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II”. In: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), pp. 182–197.
- [5] Fuda van Diggelen et al. *Emergent Heterogeneous Swarm Control Through Hebbian Learning*. <https://arxiv.org/abs/2507.11566>. 2025.
- [6] Agoston E. Eiben and Emma Hart. “If It Evolves It Needs to Learn”. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '20)*. ACM, 2020, pp. 1383–1384.
- [7] Agoston E. Eiben and James E. Smith. *Introduction to Evolutionary Computing*. 2nd ed. Springer, 2015.
- [8] Jeffrey L. Elman. “Finding Structure in Time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211.
- [9] Andrea Ferigo et al. “Evolving Hebbian Learning Rules in Voxel-Based Soft Robots”. In: *IEEE Transactions on Cognitive and Developmental Systems* 15.3 (2023), pp. 1536–1546.
- [10] Chelsea Finn, Pieter Abbeel, and Sergey Levine. “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks”. In: *International Conference on Machine Learning (ICML)*. 2017.
- [11] Dario Floreano and Joseba Urzelai. “Evolutionary Robots with On-line Self-Organization and Behavioral Fitness”. In: *Neural Networks* 13.4–5 (2000), pp. 431–443.
- [12] Wulfram Gerstner and Werner M. Kistler. “Mathematical Formulations of Hebbian Learning”. In: *Biological Cybernetics* 87.5–6 (2002), pp. 404–415.
- [13] Agrim Gupta et al. “Embodied Intelligence via Learning and Evolution”. In: *Nature Communications* 12 (2021), p. 5721.

- [14] Agrim Gupta et al. “MetaMorph: Learning Universal Controllers with Transformers”. In: *International Conference on Learning Representations (ICLR)*. 2022.
- [15] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. <https://arxiv.org/abs/1604.00772>. 2016.
- [16] Nikolaus Hansen and Andreas Ostermeier. “Completely Derandomized Self-Adaptation in Evolution Strategies”. In: *Evolutionary Computation* 9.2 (2001), pp. 159–195.
- [17] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.
- [18] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [19] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. “Planning and Acting in Partially Observable Stochastic Domains”. In: *Artificial Intelligence* 101.1–2 (1998), pp. 99–134.
- [20] Vijay R. Konda and John N. Tsitsiklis. “Actor-Critic Algorithms”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2000.
- [21] Hod Lipson and Jordan B. Pollack. “Automatic Design and Manufacture of Robotic Lifeforms”. In: *Nature* 406.6799 (2000), pp. 974–978.
- [22] Thomas Miconi, Kenneth O. Stanley, and Jeff Clune. “Differentiable Plasticity: Training Plastic Neural Networks with Backpropagation”. In: *International Conference on Machine Learning (ICML)*. 2018, pp. 3559–3568.
- [23] Thomas Miconi et al. “Backpropamine: Training Self-Modifying Neural Networks with Differentiable Neuromodulated Plasticity”. In: *International Conference on Learning Representations (ICLR)*. 2019.
- [24] Jed R. Muff et al. “Unconventional Hexacopters via Evolution and Learning: Performance Gains and New Insights”. In: *Applications of Evolutionary Computation (EvoApplications 2026)*. Vol. 16524. Lecture Notes in Computer Science. Preprint: arXiv:2505.14129. Springer, 2026.
- [25] Elias Najarro and Sebastian Risi. “Meta-Learning through Hebbian Plasticity in Random Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 20719–20731.
- [26] Yael Niv et al. “Evolution of Reinforcement Learning in Uncertain Environments: A Simple Explanation for Complex Foraging Behaviors”. In: *Adaptive Behavior* 10.1 (2002), pp. 5–24.
- [27] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the Difficulty of Training Recurrent Neural Networks”. In: *International Conference on Machine Learning (ICML)*. 2013.
- [28] Joachim Winther Pedersen and Sebastian Risi. “Evolving and Merging Hebbian Learning Rules: Increasing Generalization by Decreasing the Number of Rules”. In: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO ’21)*. ACM, 2021.

- [29] Aniruddh Raghu et al. “Rapid Learning or Feature Reuse? Towards Understanding the Effectiveness of MAML”. In: *International Conference on Learning Representations (ICLR)*. 2020.
- [30] Tim Salimans et al. *Evolution Strategies as a Scalable Alternative to Reinforcement Learning*. <https://arxiv.org/abs/1703.03864>. 2017.
- [31] John Schulman et al. “Trust Region Policy Optimization”. In: *International Conference on Machine Learning (ICML)*. 2015.
- [32] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations (ICLR)*. 2016.
- [33] John Schulman et al. *Proximal Policy Optimization Algorithms*. <https://arxiv.org/abs/1707.06347>. 2017.
- [34] Karl Sims. “Evolving Virtual Creatures”. In: *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '94)*. ACM, 1994, pp. 15–22.
- [35] Andrea Soltoggio, Kenneth O. Stanley, and Sebastian Risi. “Born to Learn: The Inspiration, Progress, and Future of Evolved Plastic Artificial Neural Networks”. In: *Neural Networks* 108 (2018), pp. 48–67.
- [36] Andrea Soltoggio et al. “Evolutionary Advantages of Neuromodulated Plasticity in Dynamic, Reward-Based Scenarios”. In: *Proceedings of the 11th International Conference on Artificial Life (ALIFE XI)*. MIT Press, 2008, pp. 569–576.
- [37] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.
- [38] Richard S. Sutton et al. “Policy Gradient Methods for Reinforcement Learning with Function Approximation”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 1999.
- [39] Christopher J. C. H. Watkins and Peter Dayan. “Q-Learning”. In: *Machine Learning* 8.3–4 (1992), pp. 279–292.
- [40] Paul J. Werbos. “Backpropagation Through Time: What It Does and How to Do It”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.
- [41] Eckart Zitzler and Lothar Thiele. “Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach”. In: *IEEE Transactions on Evolutionary Computation* 3.4 (1999), pp. 257–271.